MODULE 06

Search, Discovery & Recommendation Placements

Employee Assignment & 16-Sprint Execution Plan

Project: AI-Based Product Recommendation System

Module owner: Employee 6 / Module Owner

Project duration: 8 months | **Sprint duration:** 2 weeks | **Total sprints:** 16

1. Module Objective

Expose recommendation results consistently across shopper journeys and integrate them into React pages without blocking core shopping flows.

2. Scope of Work

• Home page recommendation rails

• Product detail related products

• Category and search-empty placements

• Cart recommendation placement

• Product similarity and related-item endpoints

• Loading, empty, timeout and error fallback UI

3. Technical Responsibilities

Key data entities/tables: Uses products, categories, recommendation_requests and recommendation_impressions; placement configuration as agreed.

API / service contract: GET /api/v1/recommendations?placement=home&limit=10; GET /api/v1/products/{product_id}/similar; placement-specific request contract.

Dependencies: Stable recommendation API response contract; catalogue endpoints; React application shell; ranking/candidate services.

4. Sprint-Wise Employee Task Plan

Each sprint is two weeks. The owner should update task status daily, raise blockers early, submit code for review, and demonstrate completed work at sprint review. Dates are to be assigned by the project manager.

Sprint	Sprint focus	Employee tasks / implementation	Sprint deliverables
Sprint 1	Placement discovery and UX	Map home, product, category, search-empty and cart placements; define expected context and fallback behaviour.	Placement matrix, wireframes and stories.
Sprint 2	API contract and UI architecture	Agree placement request/response schema, loading states, timeout handling and component boundaries.	API contract, component design and mock fixtures.
Sprint 3	React discovery shell	Create reusable product card, recommendation rail and API client structure using mock data.	Reusable UI components and frontend tests.

Sprint	Sprint focus	Employee tasks / implementation	Sprint deliverables
Sprint 4	Home recommendations	Integrate home placement with API; support guest and signed-in states and fallback content.	Home recommendation rail end-to-end.
Sprint 5	Product detail recommendations	Integrate related products and similar-item endpoint; handle missing product/context.	Product detail recommendation section.
Sprint 6	Category placement	Integrate category context and category-specific results; preserve catalogue filters and pagination.	Category recommendation placement.
Sprint 7	Search-empty placement	Show contextual recommendations when search returns no results; preserve clear search recovery path.	Search-empty state and tests.
Sprint 8	Cart placement	Integrate cart-context recommendations without interfering with cart operations or checkout-like demo flow.	Cart recommendation placement.
Sprint 9	Guest and error states	Implement no-consent, guest, no-results, unavailable product and API error states.	Complete fallback state coverage.
Sprint 10	Performance and caching	Add client-side request controls, bounded result counts and safe stale/loading behaviour; coordinate backend caching.	Frontend performance tests and cache contract.
Sprint 11	Analytics instrumentation	Attach request IDs and placement labels; ensure impression/click tracking follows event contract.	Placement event integration tests.
Sprint 12	Accessibility and responsive UX	Test keyboard navigation, semantic labels, responsive layouts and accessible contrast.	Accessibility and responsive test report.
Sprint 13	End-to-end integration	Run full shopper journeys across home, product, category, search-empty and cart placements.	E2E suite and defect fixes.
Sprint 14	Staging release	Deploy frontend integration; verify environment URLs, CORS, API timeouts and error monitoring.	Staging deployment checklist.
Sprint 15	UAT and documentation	Collect feedback; update UI guide, placement rules, API usage and known limitations.	UAT sign-off and KT page.
Sprint 16	Final handover	Demo all placements and failure states; hand over component map, environment setup and tests.	Release-ready discovery integration.

5. Module-Level Acceptance Criteria

• Placement-specific policies work as documented.

• Empty/error/timeout states are handled gracefully.

• UI does not block core browsing if recommendations time out.

• All placements pass responsive and end-to-end tests.

6. Required Testing

Unit tests: validators, business rules, transformations and core service logic. Integration tests: database migrations, API contracts, cross-module dependencies and error handling. End-to-end tests: the module's main shopper/admin/analyst journey. Negative tests: invalid inputs, unauthorized access, missing dependencies and failure/fallback paths. Performance/security tests should be added where relevant.

7. Shared Engineering Standards (Required in Every Sprint)

• Use the agreed Git branching and pull-request process; obtain at least one reviewer before merge.

• Keep configuration in environment variables; commit only a safe .env.example, never credentials or private datasets.

• Add or update automated tests with each feature; do not leave failing tests unresolved at sprint close.

• Update API contracts, schema/data dictionary, README and module notes whenever behaviour changes.

• Log useful request/job identifiers while masking personal identifiers and secrets.

• Participate in shared QA, security, Docker, CI/CD, staging, monitoring and release checks.

8. Sprint Ceremonies & Reporting

Planning: confirm sprint goal, story owners, dependencies and acceptance criteria. **Daily:** update task board and raise blockers. **Review:** demo working software, show tests and list incomplete work. **Retrospective:** record one improvement and assign an owner. **Weekly manager update:** completed tasks, in-progress tasks, blockers, risks and next steps.

9. Definition of Done

• Code is reviewed and merged to the agreed branch.

• Unit/integration tests pass and test evidence is attached to the sprint ticket.

• API/schema documentation and sample payloads are updated.

• Security and privacy implications are reviewed.

• Feature is demonstrated in a shared development or staging environment.

• Known limitations, defects and follow-up tasks are recorded.

10. Employee KT & Handover Checklist

• Explain module purpose, boundaries, architecture and dependencies.

• Walk through repository folders, entry points, data flow and key functions.

• Demonstrate one successful flow and one failure/fallback scenario.

• Run tests live and show logs/monitoring relevant to the module.

• Share setup steps, environment variable names (not secret values), migrations and commands.

• Provide API contracts, data dictionary, known issues, deployment notes and rollback/fallback steps.

• Assign a backup owner and complete handover Q&A.

Employee acknowledgement

Employee name: _____ Signature: _____ Date: _____

Manager / reviewer: _____ Review date: _____